

Real-Time Industrial Anomaly Detection — Defense Cheat Sheet

Scalable Big Data Architecture for Real-Time Industrial Anomaly Detection in Multivariate Time-Series

Thesis (say this first): Most systems pick one anomaly-detection model and live with its speed/accuracy tradeoff. This system runs a fast closed-form engine (DMD+STL) and a deep autoencoder behind one identical interface — so the tradeoff itself becomes measurable, and the empirical burst-load test proves it's real: the fast engine holds <15ms p99 latency where the autoencoder's p99 blows past 150ms.

1 · THE PROBLEM

Industrial sensors stream continuously; you must flag abnormal behavior **fast** (hard SLA: <20ms/window) and **without labels** (train only on normal data, flag deviations — unsupervised). Most papers optimize accuracy only and ignore whether the model could actually run at required speed. That gap is this paper's reason for existing.

2 · ARCHITECTURE (4 LAYERS)

- ingestion/** — asyncio producers → bounded queue → consumer batches sliding windows → thread pool scores them (never blocks intake).
- analytics/** — two engines, one shared interface `BaseAnomalyDetector` (Strategy pattern) → swappable, fair head-to-head comparison.
- evaluation/** — fuses engine error + regime distance → dynamic threshold → anomaly / not.
- benchmarks/** — measures accuracy, latency, and burst behavior; produces every number/figure in the paper.

3 · ENGINE 1: DMD + STL (FAST-TRACK)

DMD: assume $x(t+1) \approx A \cdot x(t)$. Fit A once via one SVD on normal data (no gradient descent). At inference: how wrong was A 's prediction? → `dmd_prediction_error`.

STL: split signal into seasonal + trend + residual. Residual should be small noise if normal; a big residual = anomaly → `stl_residual_energy`.

Score = 50/50 blend of both. **Why fast:** fit = one SVD (ms); score = one matrix multiply (sub-ms), no iteration ever.

7 · Key Numbers (memorize these)

Metric	NASA C-MAPSS (real)	TCM5 (synthetic)
AUC-ROC (DMD / LSTM AE)	0.647 / 0.597	1.000 / 0.993
F1 (DMD / LSTM AE)	0.077 / 0.194	0.680 / 0.716
Nominal p99 latency (DMD / LSTM AE)	0.48ms / 2.74ms	0.20ms / 3.17ms
Extreme-burst p99 (DMD / LSTM AE)	14.0ms / 179.5ms	1.5ms / 153.7ms
Extreme-burst SLA violations (DMD / LSTM AE)	1/50 / 22/49	0/58 / 25/45
Engine fit/init time	DMD: 18–156ms AE: 0.2–5.8s (target was <20 min)	

8 · ANTICIPATED Q&A

Q: Why harmonic mean, not average?

A: Can't be fooled by one good signal masking one bad one — needs *both* low.

Q: Why is C-MAPSS's F1/AUC so much lower than TCM5's?

A: Gradual continuous degradation (fuzzy label boundary) vs. TCM5's discrete injected anomaly (clean boundary). Not a bug — an honest property of the task; AUC≈1.0 there would be suspicious.

4 · ENGINE 2: AUTOENCODER

Neural net trained to compress & reconstruct *normal* windows only. Good at reconstructing normal shapes, bad at anomalous ones (never saw them) → high reconstruction MSE = anomaly.

LSTM variant: true sequence model, any window length. **Dense variant:** flattens window, cheaper/faster but locked to one window length (dimension-mismatch guard needed — see Bug Audit B).

Result: Dense AE ≈ DMD's latency, keeps LSTM's accuracy → the LSTM's *recurrence*, not "being a neural net," is what costs the latency.

5 · EVALUATION ENGINE

Composite score (harmonic mean of distance-to-normal-regime d and engine error e):

$$\text{score} = 2 \cdot d \cdot e / (d + e + \epsilon), \epsilon = 1e-8$$

Harmonic mean — not average — because it's dominated by the *smaller* input: a window only looks normal if **both** signals agree. One good signal can't mask one bad one.

Dynamic threshold: rolling **mean+3σ** of last 500 "passed" scores. Bug found empirically: if baseline drifts, every new-normal score exceeds the stale threshold and never updates it → stuck forever. Fix: after 50 consecutive flags, force-recalibrate from the recent run. This alone took TCM5 F1 **0.249 → 0.391**.

6 · BUG AUDIT (5 CATEGORIES, 56 TESTS)

- Concurrency:** `ThreadPoolExecutor`'s internal queue is *unbounded* → would leak memory under load. Fixed with a semaphore that bounds in-flight windows & backpressures the whole chain.
- Dimension mismatch:** Dense AE / DMD now raise clear errors on wrong channel count or window length, instead of crashing inside `numpy/PyTorch`.
- PyTorch leakage:** every inference wrapped in `torch.no_grad() + .eval()` — verified via a grad-spy test and a determinism test.
- Div-by-zero:** harmonic mean & threshold σ both floored at $\epsilon=1e-8$.
- NaN/Inf:** sanitized at ingestion with a causal running-mean impute, flagged via `was_sanitized`.
- Found only by running the full experiment:** C-MAPSS "normal" train split secretly contained near-failure data (trimmed to match test's anomaly horizon) + the threshold-drift bug above.

Q: Why drop 60%+ of packets in the extreme burst?

A: Deliberately 5–8× beyond provisioned capacity, to find the breaking point. Point isn't "never drop" — it's "degrade safely (bounded memory, no crash) instead of falling over."

Q: Why 2 datasets, one real one synthetic?

A: NASA = real, messy, gradual degradation. TCM5 = controlled, reproducible, license-free ground truth to validate the pipeline itself works before trusting messier real data.

Q: What's the actual novel contribution?

A: Not a new model — a *system*: dual engines under one interface + co-designed concurrency/latency as measured SLA properties + a bug audit as part of the empirical result, not just cleanup.

Q: Biggest limitation / future work?

A: Distance term uses raw values, not trend-aware — hurts gradual-degradation tasks specifically. Also: adaptive engine selection (escalate to AE only on borderline DMD scores).